Adamson University College of Science
IT and IS Department

Course Code	ITCD 103	Section	29104
Course Details	Software Development for Enterprise Systems	Date	
Names	Fernandez, Aaron Clifford Gardoce, Allan Mark Portes, John Dale Salazar, Kaede Sotelo, Andria Faye Vegerano, Miguel Joseph	Hands-On Laboratory	

SDLC Phase 3
Hands-on Laboratory Exercise 3

Objective: The objective of Phase 3 The Implementation phase (Phase 3) of the SDLC is where your project ideas take shape through coding:

1 Labs provide a platform for students to actively apply the theoretical concepts learned in previous phases (requirements and design).
2 Labs offer a controlled environment to practice coding techniques, syntax, and problem-solving approaches.
3 Labs that can allow students to explore functionalities beyond the core requirements.
4 Labs act as stepping stones towards completing the larger project.
5 Labs provide a crucial link between theoretical knowledge and real-world application.

1.1 Align Labs with Project Goals

1.1.1 Identify Key Concepts

Pinpoint the fundamental programming concepts your project aims to develop.

The Computer Repair Shop Management System development focuses on the following fundamental programming concepts:

1. **Object-Oriented Programming (OOP)**
   • Data modeling through objects (Customer, Repair Ticket, Service History)
   • Entity relationships (one-to-many associations)
   • Inheritance using Salesforce standard objects

2. **Database Management**
   • Relational database design with proper entity relationships
   • SOQL queries for data retrieval
   • Data integrity through validation rules and constraints

3. **User Authentication and Security**
   • Role-based access control (Admin, Receptionist, Technician)
   • Profile and permission set configuration
   • Field-level and object-level security

4. **Business Process Automation**
   • Workflow automation using triggers and process builder

- Event-driven programming for status updates
- Automated notification systems

5. **User Interface Development**
   - Form design for data entry
   - Dashboard creation for metrics visualization
   - Responsive design principles

6. **API Integration**
   - Email-to-Case integration
   - REST API concepts for external communication
   - Web service integration

7. **Cloud Computing**
   - SaaS architecture understanding
   - Scalability and multi-tenancy concepts
   - Cloud-based data management

8. **Error Handling and Testing**
   - Exception handling in Apex
   - Input validation and sanitization
   - Unit testing and debugging strategies

9. **Reporting and Analytics**
   - Data aggregation and calculated fields
   - Report generation and visualization
   - Business intelligence dashboards

These concepts provide the technical foundation for implementing a fully functional enterprise repair management system using Salesforce's development platform.

1.1.2 Break Down Functionality

Decompose the project's functionalities into smaller, learnable modules.

To effectively transition from design to implementation, the system is broken down into smaller functional components. Each represents a specific responsibility within the system and is designed to be developed and tested independently while still working cohesively within the overall Salesforce architecture.

This modular approach ensures clarity in development, easier debugging, and better scalability as the system grows.

I. User Authentication and Role Management

This component handles user access to the system, ensuring that each user interacts only with features relevant to their assigned role. Authentication is managed through Salesforce’s built-in login system, while access control is enforced using Profiles and Permission Sets to maintain security and proper authorization.

Core Functions:

- User authentication
- Role-based access control (Admin, Receptionist, Technician)

- Permission and visibility management

II. Customer Management

This component manages all customer-related data and serves as the foundation for linking repair records and service history. It serves as a central data entity that other components depend on for consistency and accurate information retrieval.

Core Functions:

- Creating and updating customer records
- Searching customers by name or phone number
- Viewing associated service history

III. Repair Ticket Management

This component handles the creation and management of repair tickets, which represent the core transactions within the system. It tracks device issues and organizes repair workflows from initiation to completion.

Core Functions:

- Creating repair tickets with device and issue details
- Generating unique ticket identifiers
- Assigning technicians to repair tasks

IV. Status Tracking and Workflow Automation

This component manages the progression of repair tickets and automates system behavior based on status changes. It reduces manual work by triggering actions automatically when specific conditions are met.

Core Functions:

- Updating repair status (Received, In Progress, Awaiting Parts, Completed, Released)
- Allowing technicians to add notes and updates
- Triggering automated workflows using Apex Triggers

V. Notification System

This component is responsible for handling communication between the system and customers, ensuring that users receive timely updates regarding their repairs. Notifications are automatically triggered based on system events.

Core Functions:

- Sending email notifications when repairs are completed
- Integrating with Email-to-Case functionality
- Triggering notifications based on status changes

VI. Dashboard and Reporting

This component provides visibility into system operations by presenting summarized data and performance metrics. It supports decision-making by giving shop owners insights into repair activities and overall performance.

Core Functions:

- Displaying real-time repair statistics
- Generating reports and summaries
- Providing operational insights

VII. Service History Management

This component maintains a complete record of past repair activities associated with each customer. It ensures that historical data is organized and easily accessible for reference and analysis.

Core Functions:

- Storing and retrieving repair history
- Linking multiple repair records to a single customer
- Maintaining consistent data relationships

1.1.3 Map Labs to Modules

Design your project ideas focusing on each module, gradually building towards the entire project.

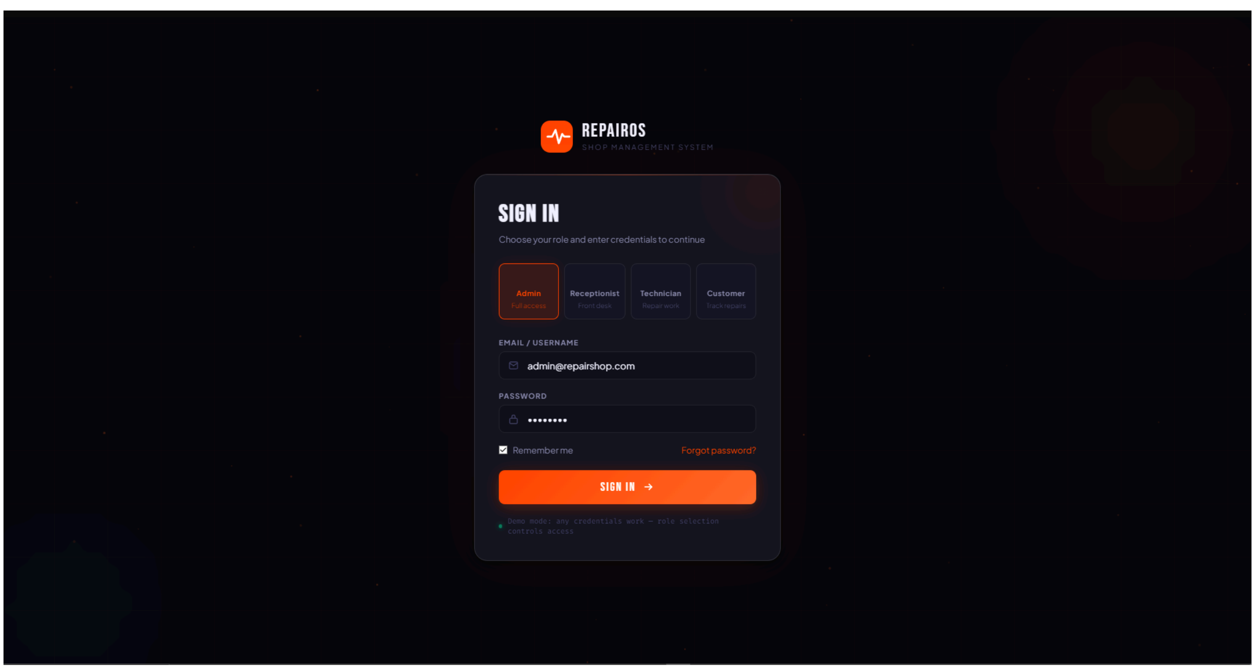

Figure 1. **User Authentication and Role Management Module**

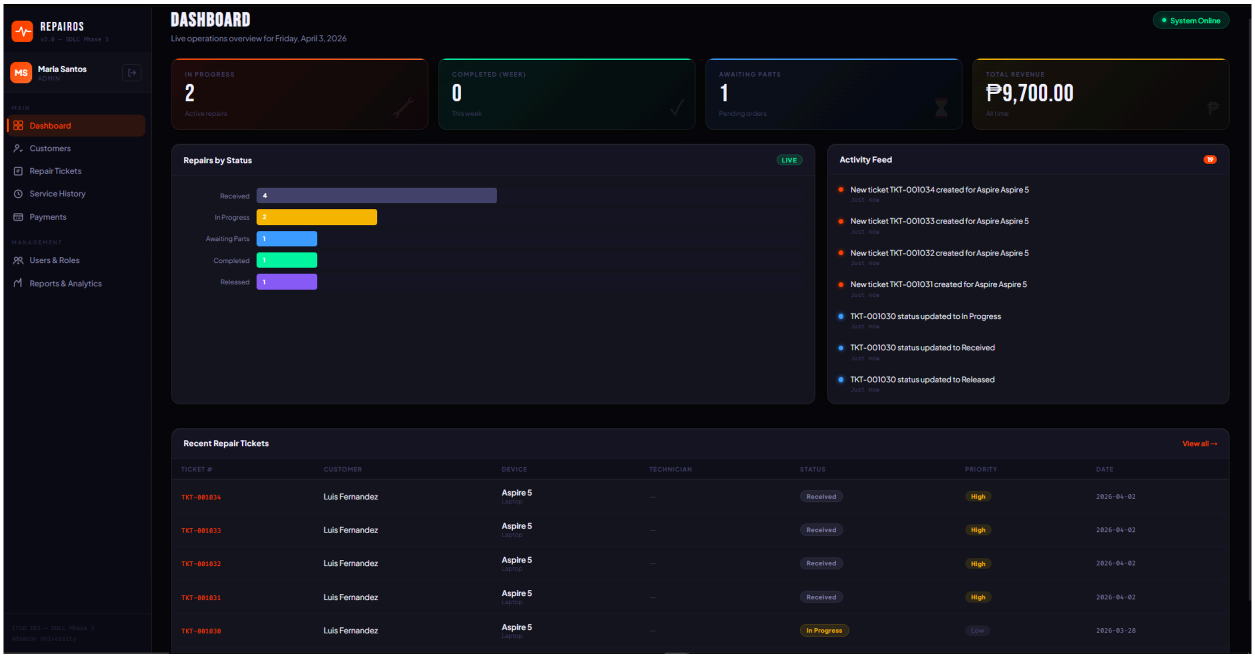

Figure 2. **Admin Dashboard Module**

Figure 3. **Admin Report Analytics Module**

Adamson University College of Science
IT and IS Department

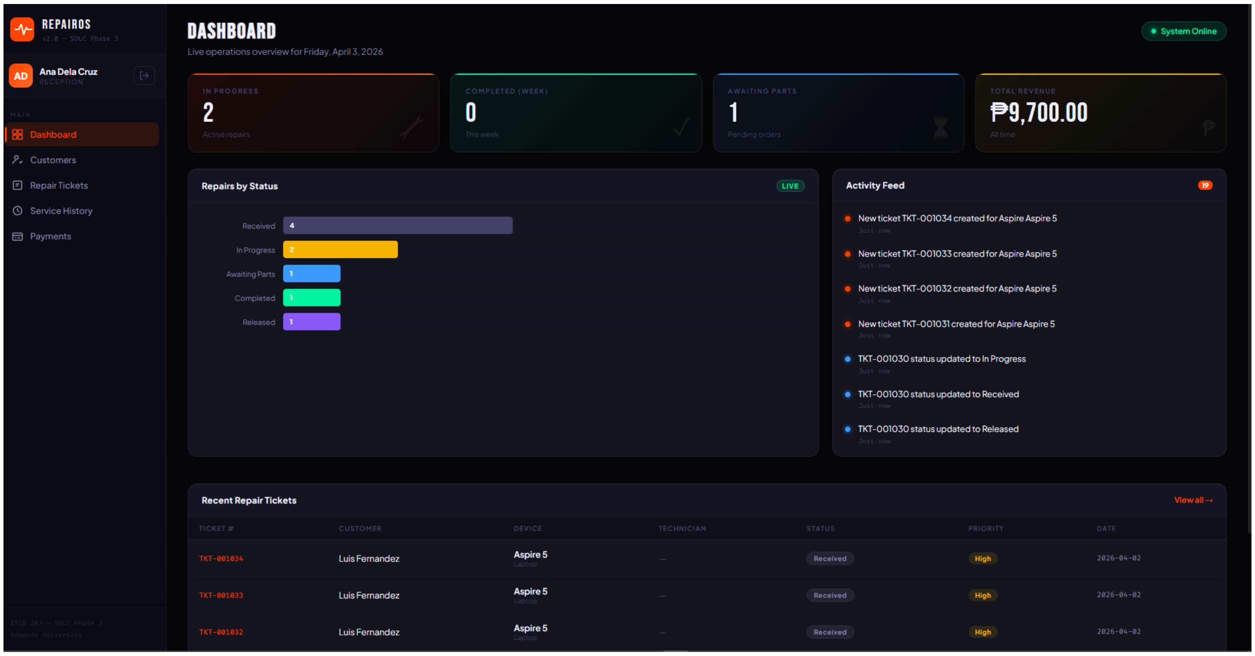

Figure 4. **Receptionist Module**

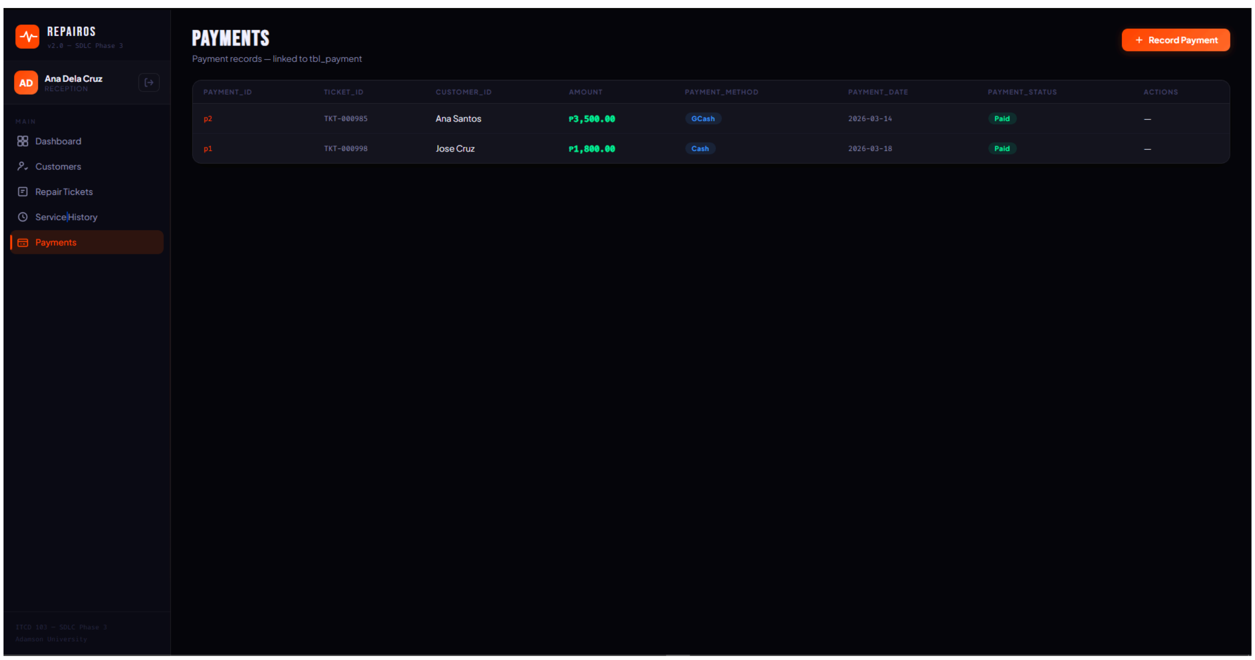

Figure 5. **Receptionist Payments Module**

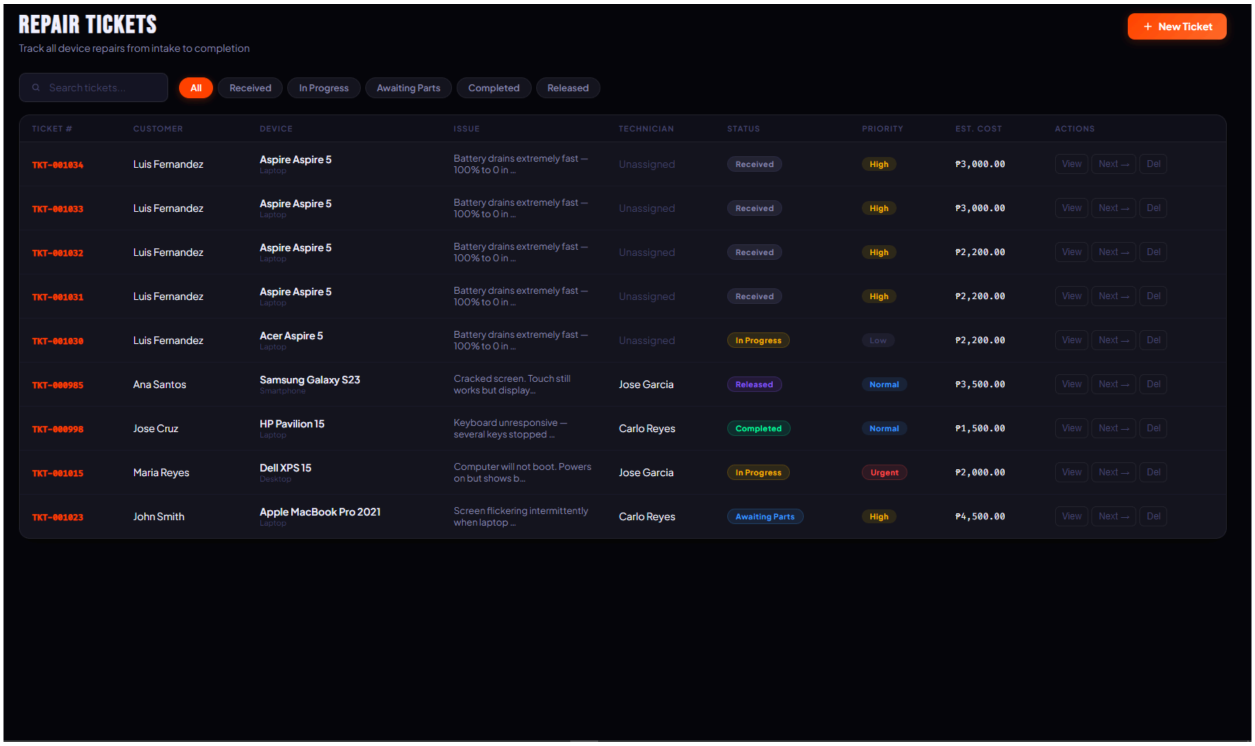

Figure 6. **Technician Tickets Module**

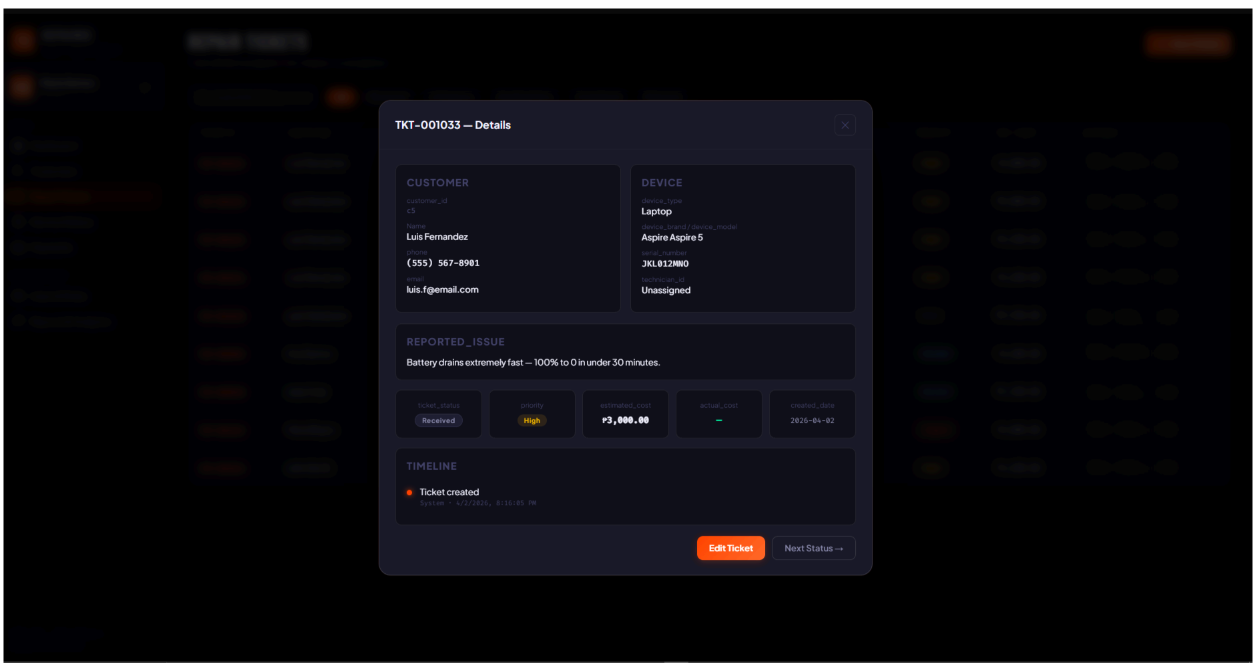

Figure 7. **Technician Update Ticket Module**

Adamson University College of Science
IT and IS Department

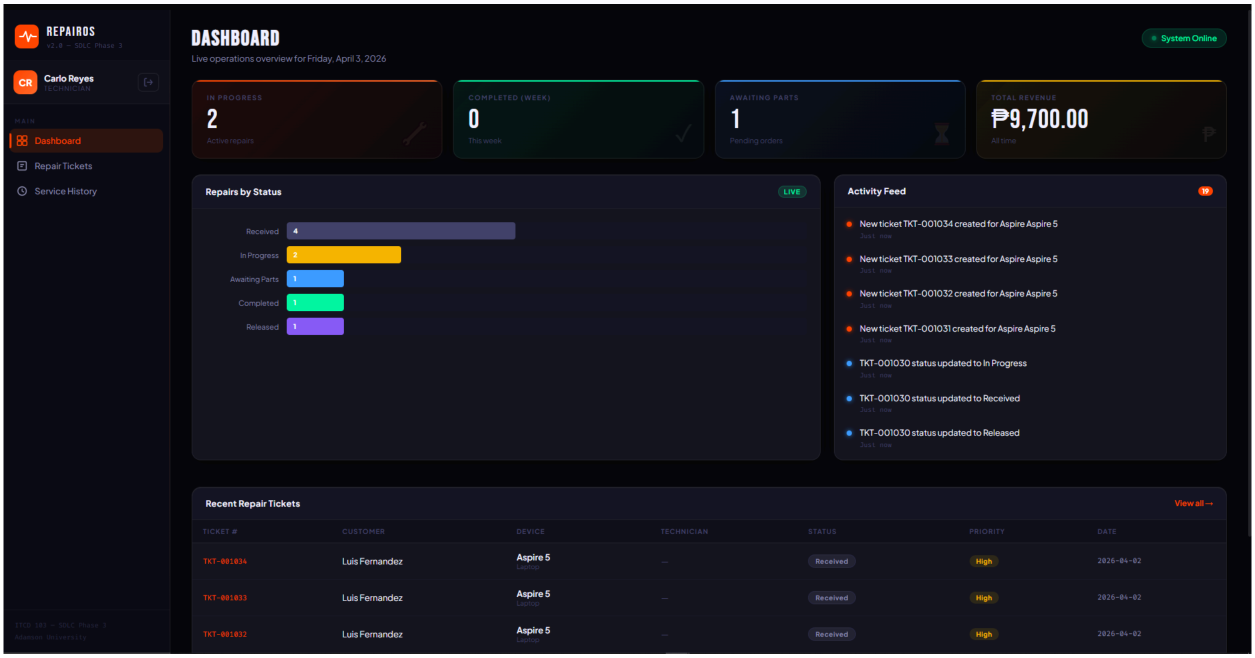

Figure 8. **Technician Dashboard Module**

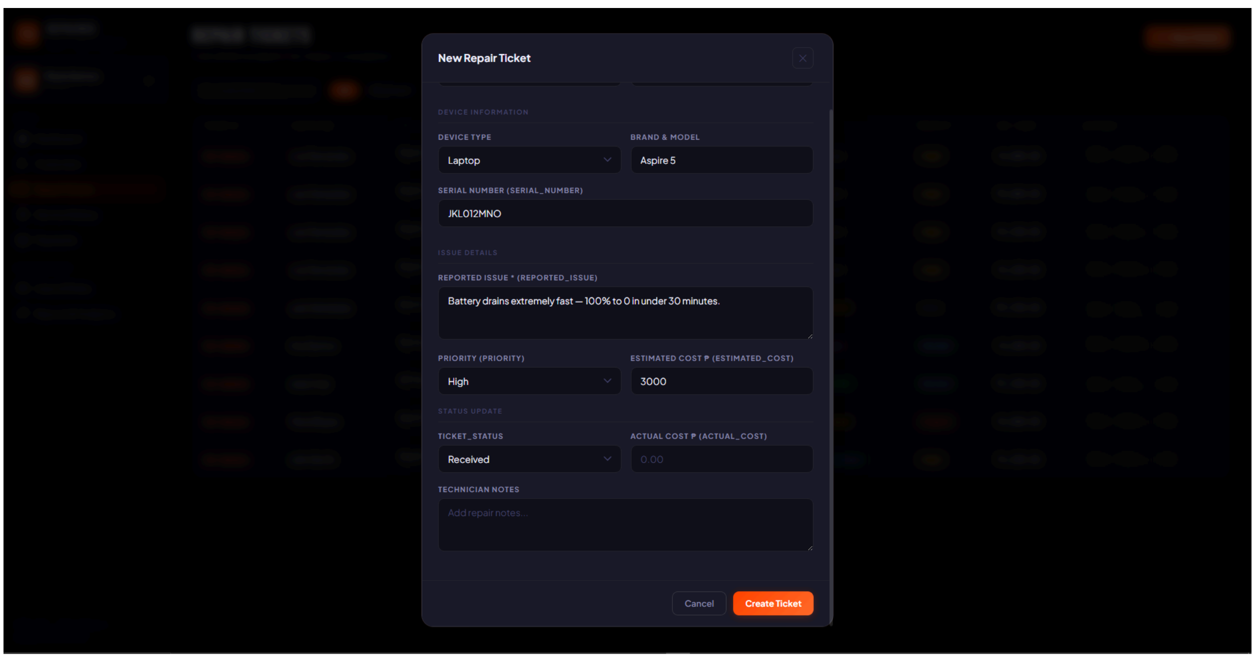

Figure 9. **User Authentication and Role Management Module**

Adamson University College of Science
IT and IS Department

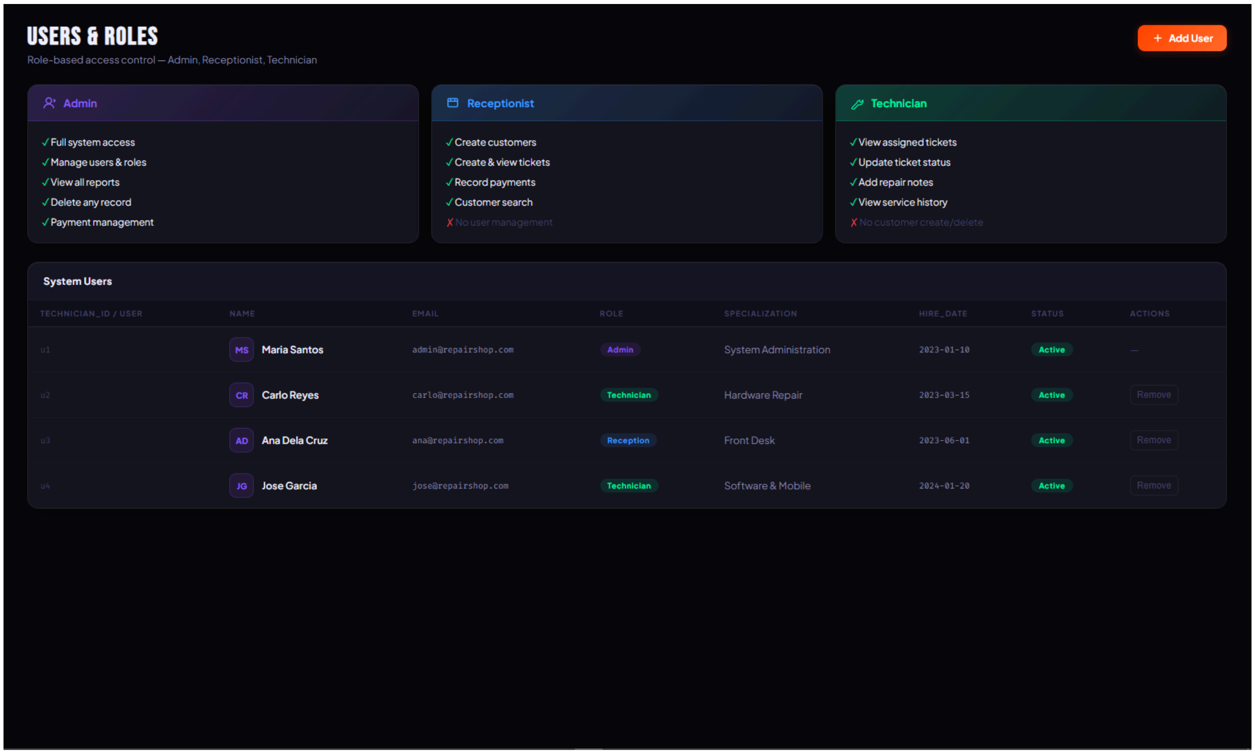

Figure 10. **Admin User & Roles Module**

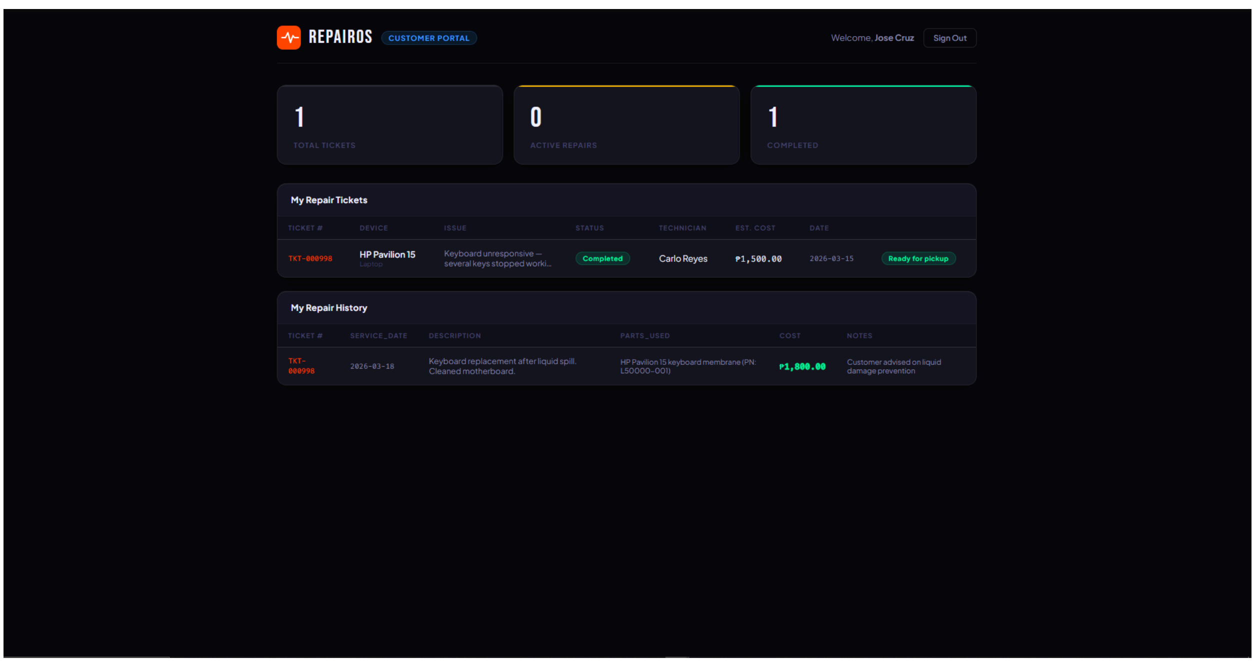

Figure 11. **Customer Module**

Adamson University College of Science
IT and IS Department

1.2 Craft Engaging Learning Experiences

1. Show your step-by-step instructions with visuals or screenshots/diagrams for easy understanding.

Step 1. Secure Authentication and Role Selection

- Users begin by accessing the sign-in portal. The system utilizes Role-Based Access Control (RBAC), allowing users to select their specific designation (Admin, Receptionist, Technician, or Customer) before entering credentials.

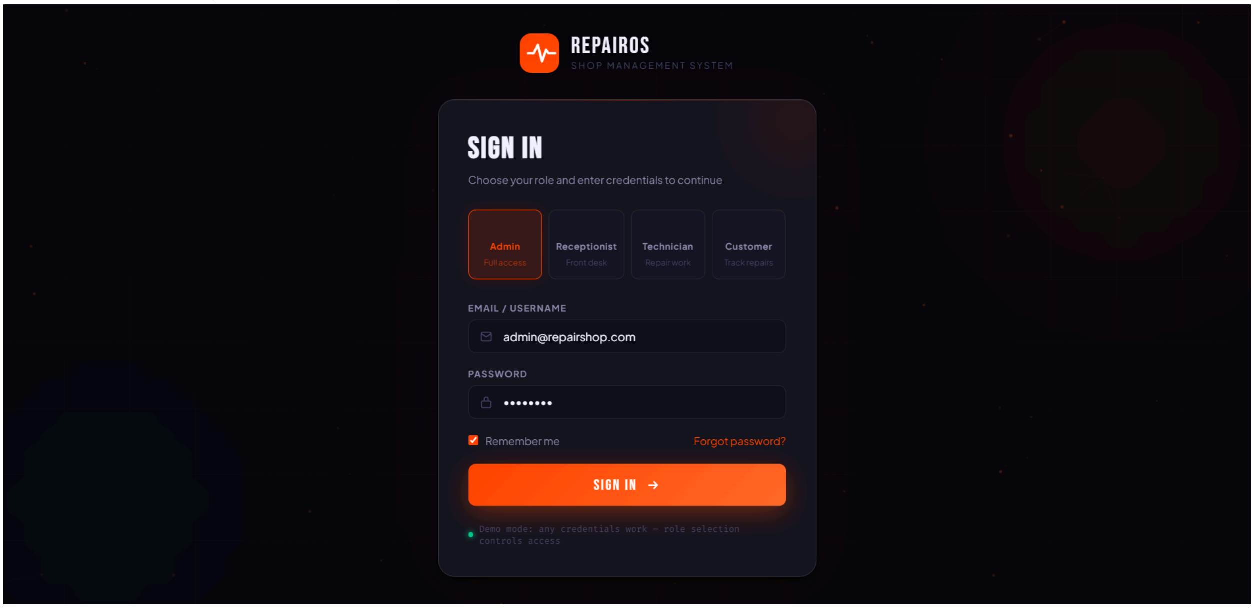

Figure 12. **Select Role and enter credentials to access full system management.**

Step 2: Monitoring Operations via the Admin Dashboard

- Once logged in, the Admin is greeted by a real-time dashboard. This provides a high-level overview of system health and repair activities.

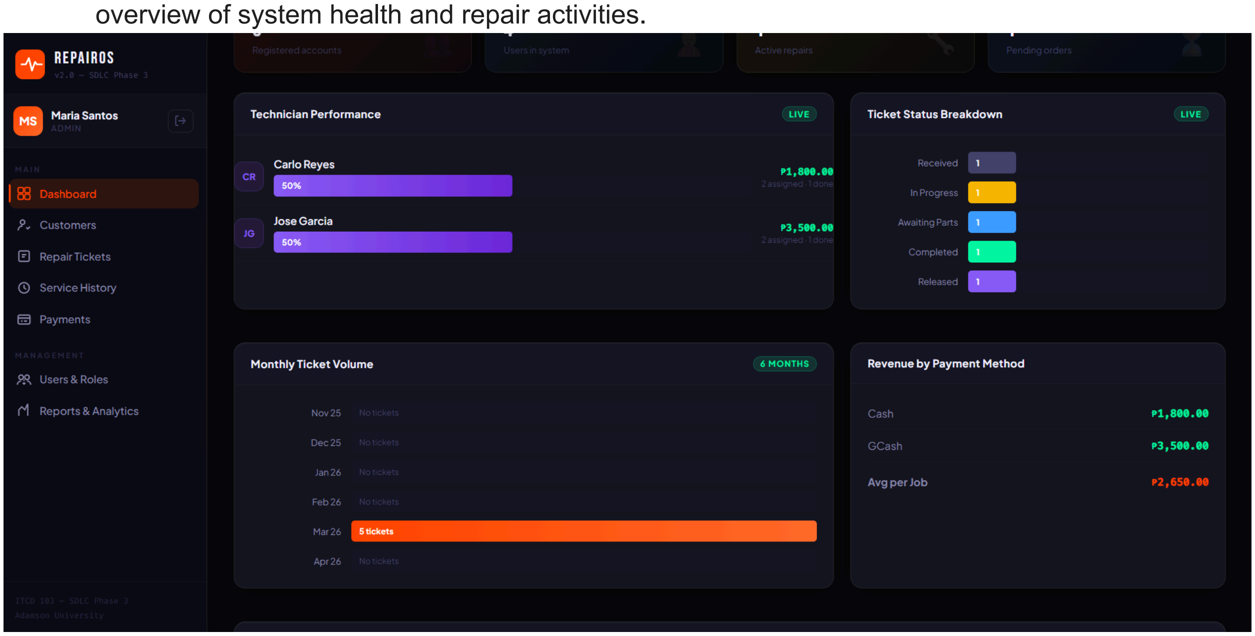

Figure 13. **Review the "Ticket Status Breakdown" bar chart to identify bottlenecks, such as tickets marked "Awaiting Parts".**

Step 3: Creating a New Repair Ticket (Receptionist/Technician)

- When a customer brings in a device, a new repair ticket is generated to track the workflow from initiation to completion.

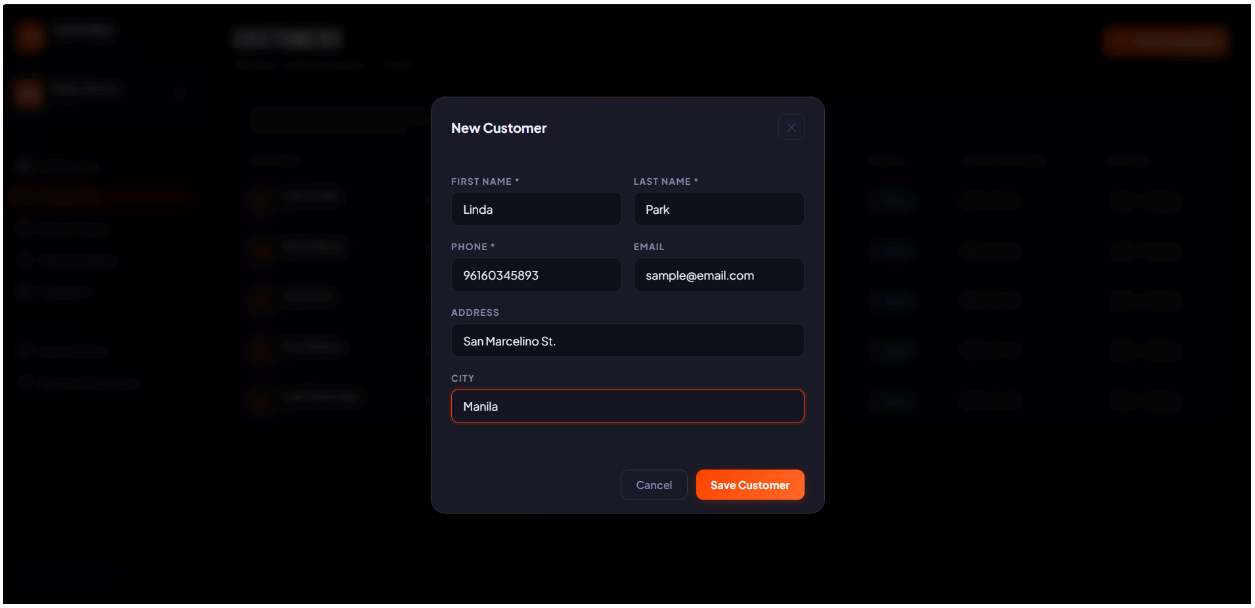

Figure 14. **Creating New Customer**

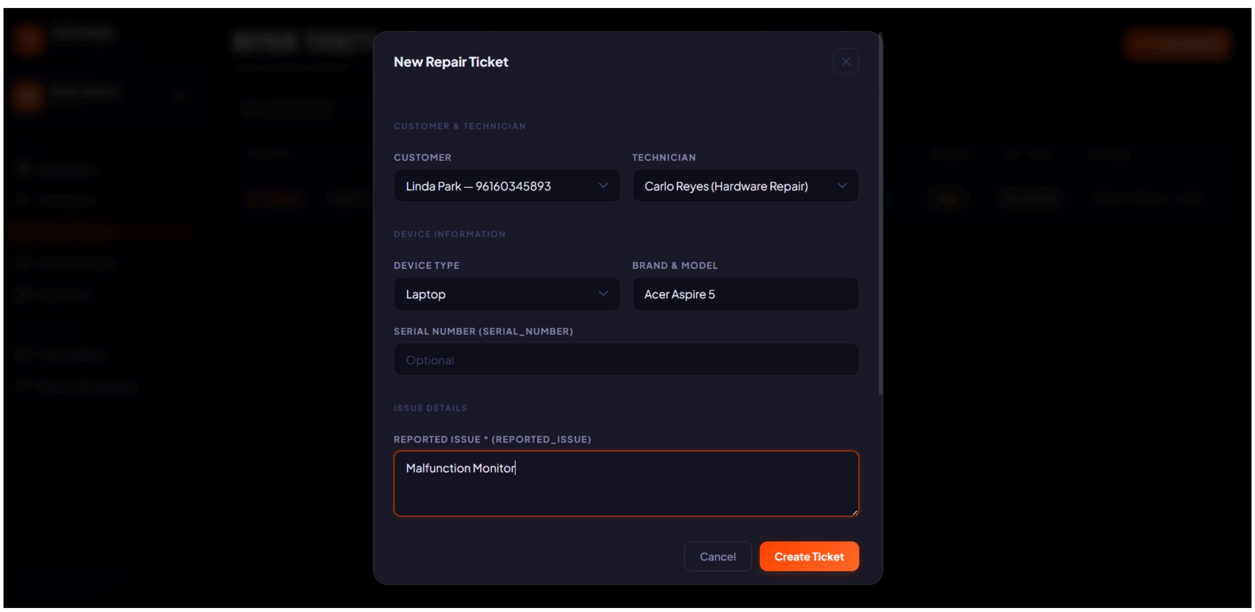

Figure 15. **Creation of New Repair Ticket**

Step 4: Managing Technical Updates and Timelines

- Technicians use the "Repair Tickets" module to update the status of assigned tasks and add technical notes for the service history.

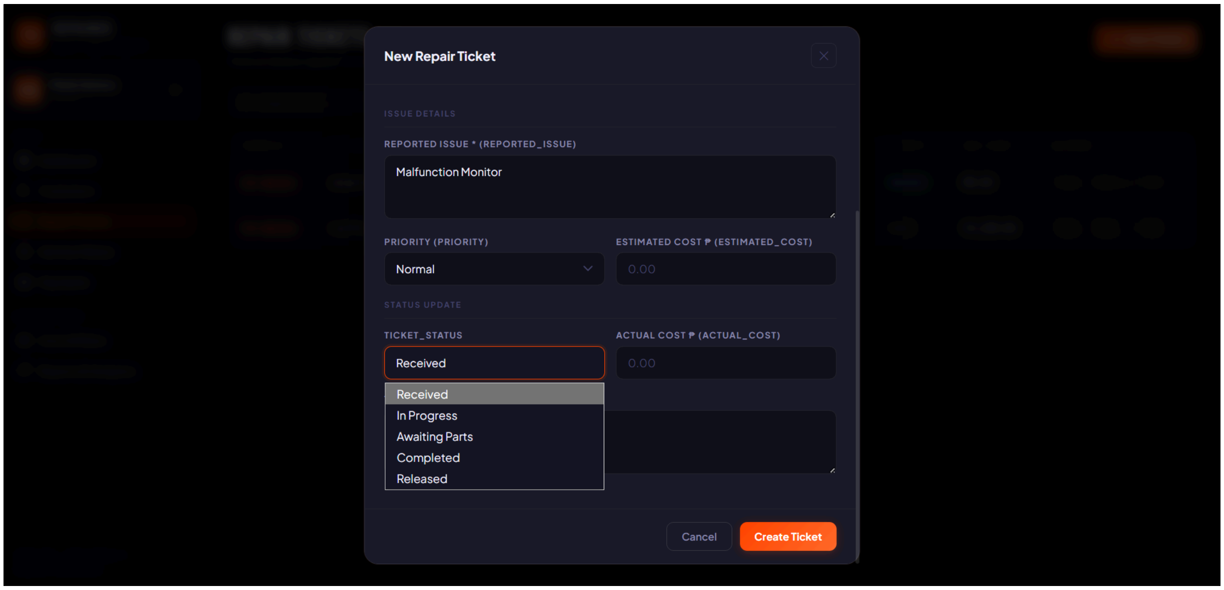

Figure 16. **Editing of Ticket Status**

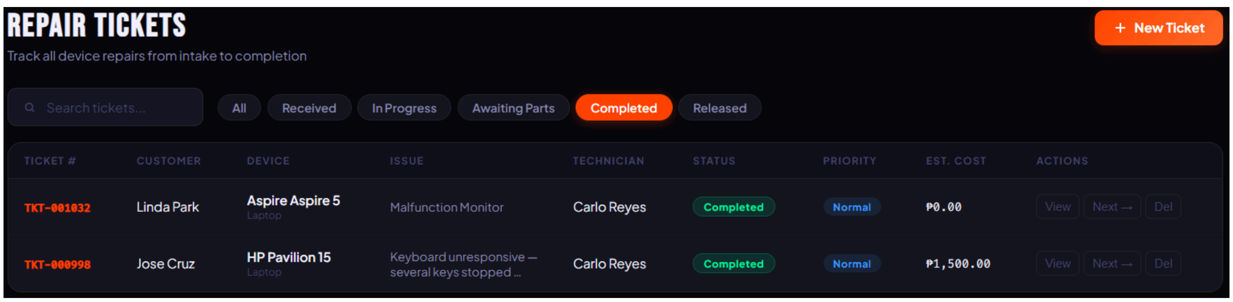

Figure 17. **Repair Ticket Completion**

Step 5: Reviewing Performance and Analytics

- Shop owners or Admins use the "Reports & Analytics" module to make data-driven decisions based on revenue and ticket volume.

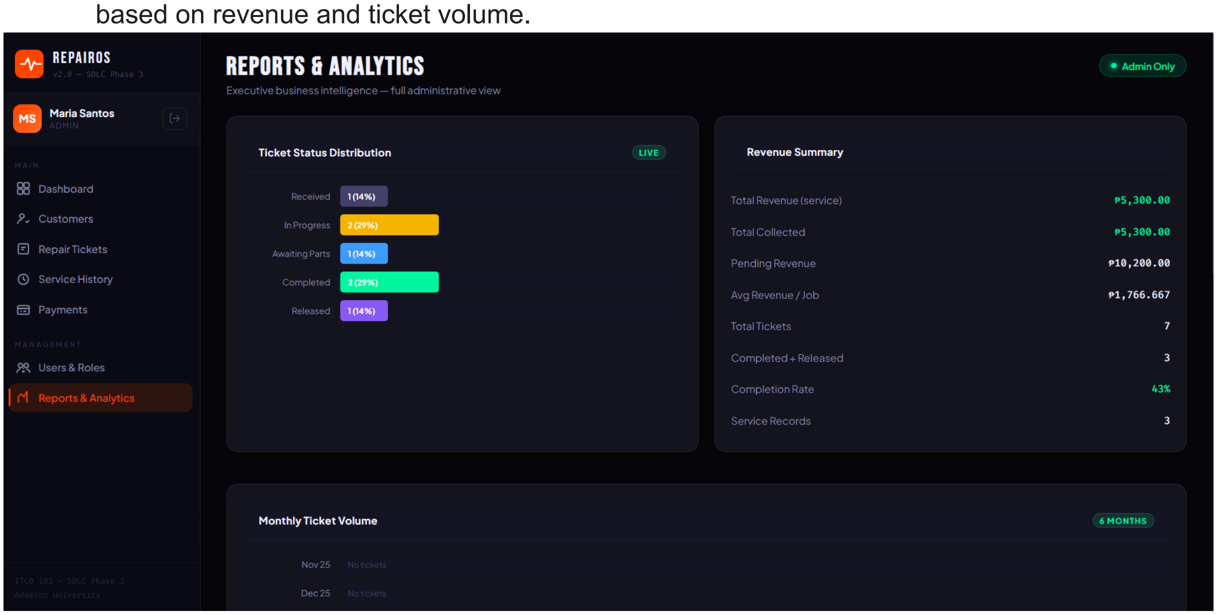

Figure 18. **Analyze trends through Reports & Analytics**

2. Show the pre-written code snippets to illustrate specific concepts and functionalities.

   <button class="role-tile active" data-role="admin" onclick="selectRole(this,'admin')">
   Explanation:
   This code allows users to select their role before logging in. The data-role attribute defines the role (Admin, Receptionist, etc.), while selectRole() updates the system state. This implements Role-Based Access Control (RBAC).

   <button class="btn-signin" onclick="doLogin()">
   Explanation:
   Triggers the doLogin() function when clicked. This function handles authentication and redirects the user to the correct interface depending on their role.

   <a href="#" class="sb-link active" data-page="dashboard" onclick="return navTo('dashboard',this)">
   Explanation:
   This enables navigation between pages without reloading. The navTo() function switches visible sections dynamically, making the system behave like a Single Page Application (SPA).

   <section id="sec-dashboard" class="sec active">
   Explanation:
   Each system feature is placed inside a <section>. Only one section is visible at a time using the active class. This organizes the UI into manageable modules.

   <input type="text" id="cust-search" oninput="renderCustomers()"/>
   Explanation:
   This enables real-time searching. As the user types, renderCustomers() updates the table instantly, improving usability and efficiency.

   <button class="btn-primary" onclick="openNewTicket()">
   Explanation:
   Opens a modal form where users can create a new repair ticket. This is a key feature for managing repair workflows.

   <button class="pill" onclick="filterStatus('In Progress',this)">
   Explanation:
   Filters repair tickets by status (e.g., In Progress, Completed). Helps staff quickly track work progress.

   <tbody id="hist-tbody"></tbody>
   Explanation:
   This is where service history data is dynamically inserted using JavaScript. It represents records similar to a database table.

   <select class="field-input" id="p-method">
   <option>Cash</option>
   <option>GCash</option>
   </select>
   Explanation:
   Allows users to select payment methods. Supports real-world options, especially common in the Philippines like GCash.

   <select class="field-input" id="u-role">
   <option value="admin">Admin</option>
   Explanation:

Defines the role of a system user. This determines what features they can access.

3. Incorporate mini-challenges within project ideas to test comprehension and encourage problem-solving.

Challenge 1

Enhance system feature
Task:
• Create a notification system that sends an email when:
• Status = Completed
Bonus:
• Customize the email message

Challenge 2

Real-world problem
Scenario:
• A customer has multiple repair records.
Task:
• Display all previous repairs under one customer
Expected Output:
Customer: Juan Dela Cruz

Repair History:
- RT-00120 (Completed)
- RT-00115 (Released)
- RT-00110 (Completed)

Challenge 3

User Authentication & Security
Task:
• Prevent Receptionists from accessing Admin features.
Guide Questions:
• How will you check the user role?
• What happens if unauthorized access is attempted?
Expected Output:
Access Denied: do not have permission to view this page.

Challenge 4

Modify functionality
Task:
• Automatically change status to “Completed” when technician adds final notes
Hint:
• Use a trigger or workflow automation

Note: Present your idea using the 1.1 and 1.2 materials as an outline (print screen or place your ideas underneath each item number with details).

Conclusion:

The SDLC Phase 3 (Implementation Phase) helped us turn our system design into a working application through actual coding and development. By breaking the system into smaller modules, we were able to build and understand each feature more easily and ensure that all components work together effectively.

Using step-by-step instructions, code snippets, and mini-challenges made the learning process more engaging and improved our problem-solving and programming skills. Overall, this phase served as an important bridge between planning and execution, preparing us for testing, deployment, and further system improvements.